Introduction to the Theory of Computation

Lecture 18. Reductions

Neo (Yuan) Li

Nov 2026

Fudan University

yuan_li@fudan.edu.cn

Why Do We Need Reductions?

- **Upper bounds:** reduce X to Y; a fast algorithm for Y gives one for X.
- **Hardness:** reduce Y to X; if Y is hard, so is X.
- **Complete problems:** the hardest problems in a class. Solve one efficiently, and the whole class falls.

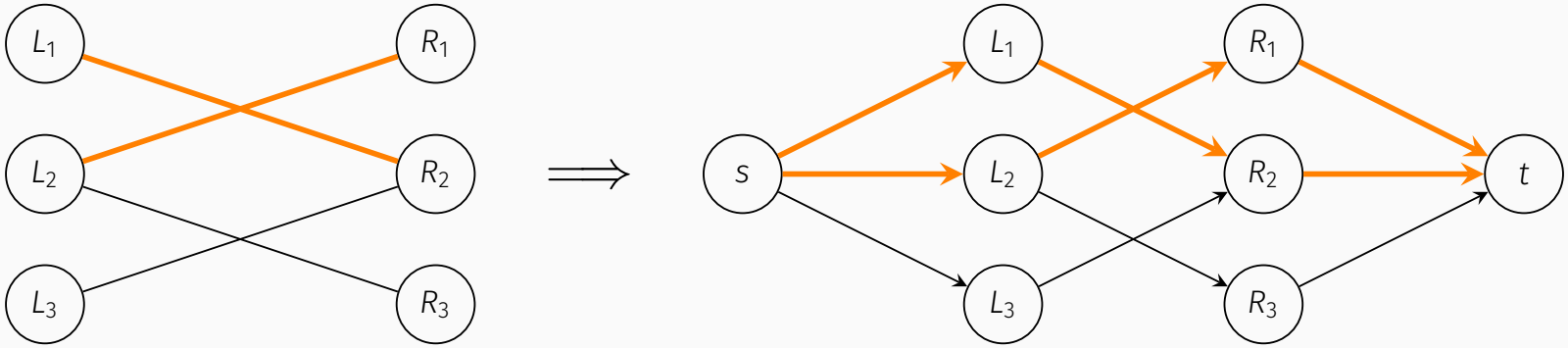
Example: Chickens and Rabbits

A cage has 8 heads and 26 legs. How many chickens, how many rabbits?

- **Primary school trick:** every chicken stands on 1 leg, every rabbit on 2. Legs on the ground: $26/2 = 13$, so $13 - 8 = 5$ rabbits and 3 chickens.
- **Middle school:** *reduce* to a linear system.

$$c + r = 8, \quad 2c + 4r = 26 \quad \Longrightarrow \quad c = 3,\ r = 5.$$

Example: Reduce Maximum Matching to Maximum Flow



Add a source s and a sink t; every edge has capacity 1.

Max matching size $=$ max flow value (here, 2).

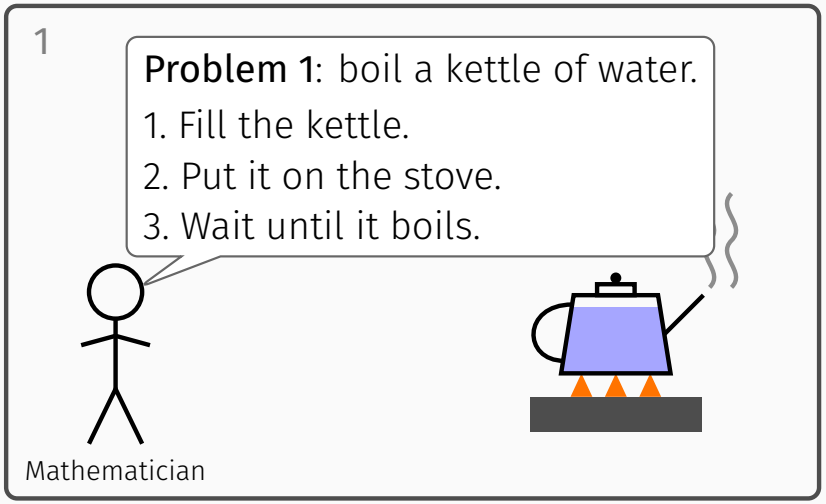
Example: Multiplication via Squaring

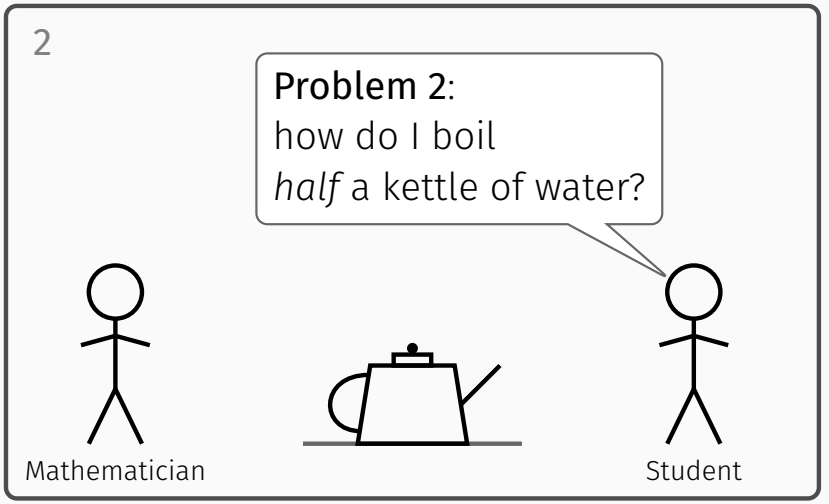
Reduce Multiplication to Squaring:

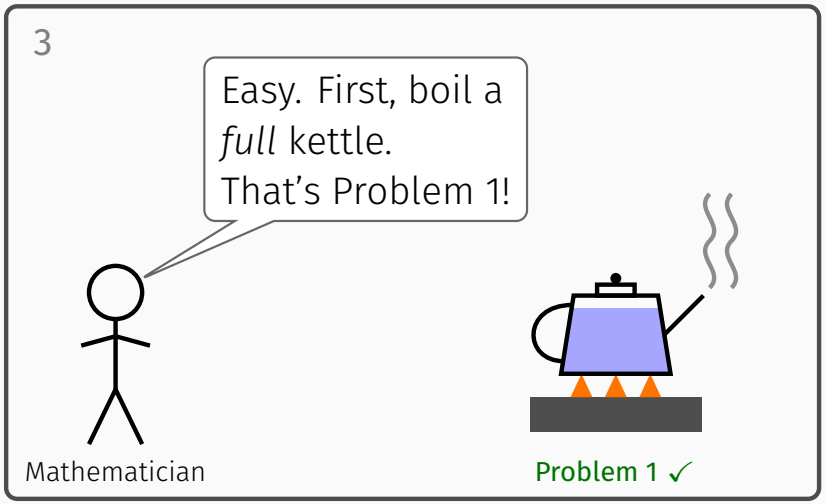
$$a \times b = \frac{(a+b)^2 - a^2 - b^2}{2}.$$

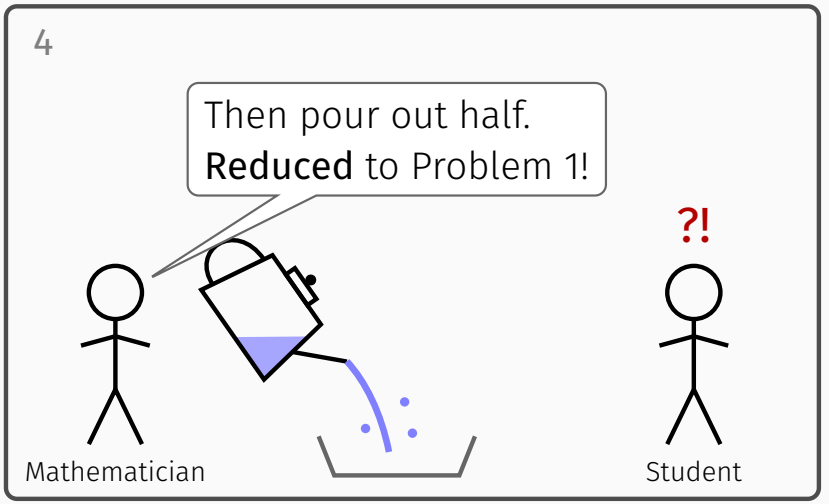
The squaring subroutine is called three times.

Reductions, the Mathematician's Way









Types of Reduction

	computable	poly-time
Many-one (one call)	$L_1 \leq_m L_2$	$L_1 \leq_p L_2$ (Karp)
Turing (many calls)	$L_1 \leq_T L_2$	$L_1 \leq_T^p L_2$ (Cook)

NP-completeness uses Karp reductions $\leq_p$.

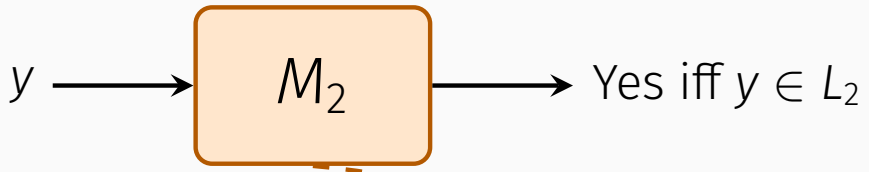
Many-One Reduction: Formal Definition

Definition 1 (Many-one reduction)
L_1 is *many-one reducible* to L_2, written $L_1 \leq_m L_2$, if there is a computable function $\varphi : \{0,1\}^* \to \{0,1\}^*$ such that for every x,

$$x \in L_1 \iff \varphi(x) \in L_2.$$

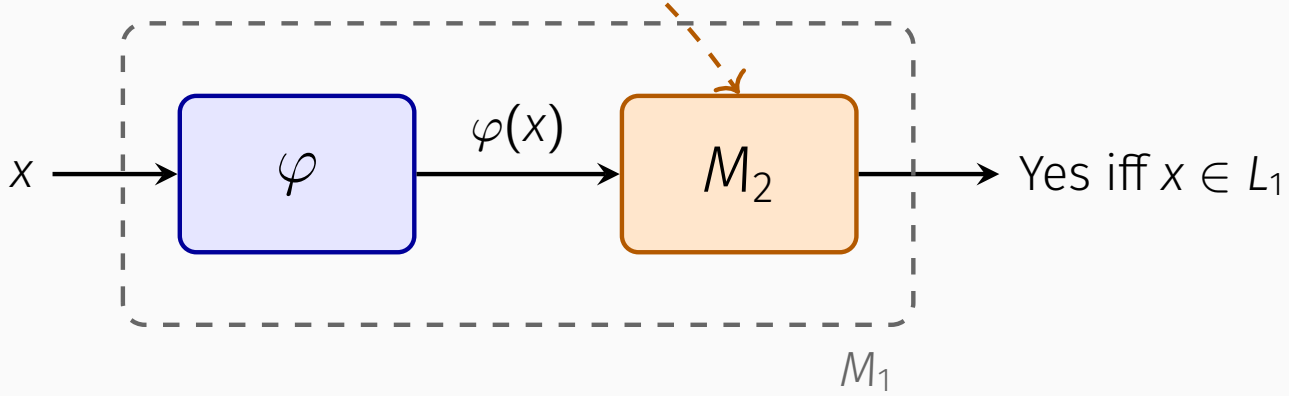
Reduction from L_1 to L_2

Given: a decider for L_2



subroutine

Build: a decider M_1 for L_1



Correct because $x \in L_1 \iff \varphi(x) \in L_2$.

Basic Properties of $\leq_m$

Proposition 1

1. $L \leq_m L$.
2. $L_1 \leq_m L_2 \iff \overline{L_1} \leq_m \overline{L_2}$.
3. *If* $L_1 \leq_m L_2$ *and* $L_2 \leq_m L_3$, *then* $L_1 \leq_m L_3$.
4. *If* $L_1 \leq_m L_2$ *and* L_2 *is decidable, then* L_1 *is decidable. Equivalently, if* L_1 *is undecidable, then* L_2 *is undecidable.*

Poly-Time Many-One Reduction: Karp Reduction

Definition 2 (Karp reduction)
L_1 is *polynomial-time reducible* to L_2, written $L_1 \leq_p L_2$, if there is a function $\varphi : \{0,1\}^* \to \{0,1\}^*$ such that

1. φ is computable in polynomial time;
2. for every x, $x \in L_1 \iff \varphi(x) \in L_2$.

NP-Complete Problems

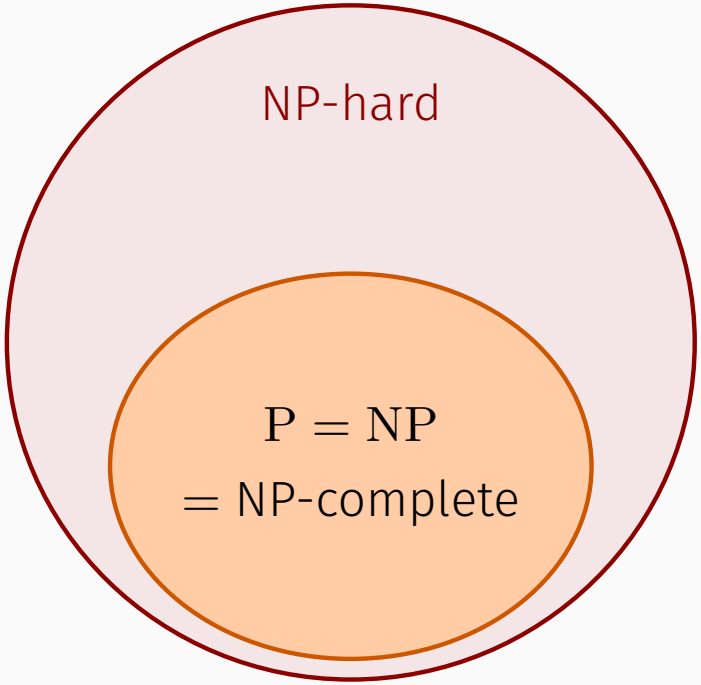
Definition 3 (NP-hard)
L is *NP-hard* if $K \leq_p L$ for every $K \in \mathrm{NP}$.

Definition 4 (NP-complete)
L is *NP-complete* if $L \in \mathrm{NP}$ and *L* is NP-hard.

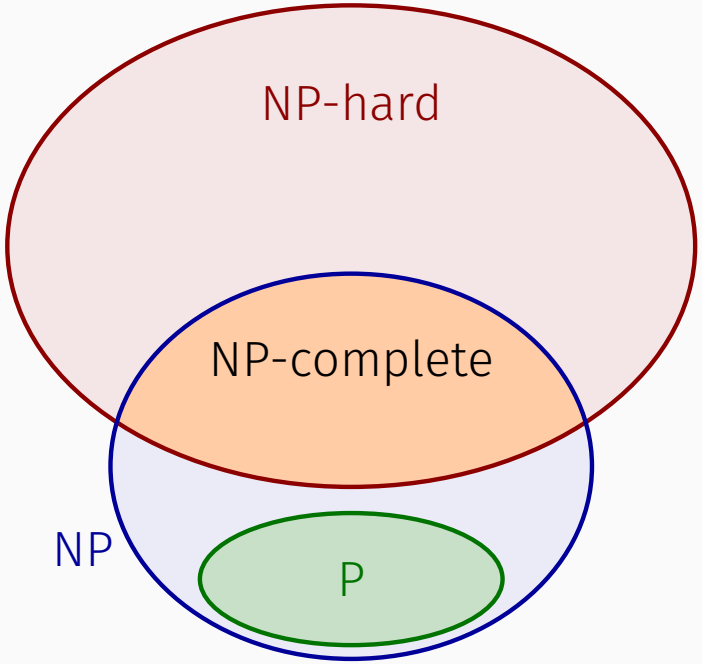
Proposition 2
Let L be NP-complete. Then $L \in \mathrm{P} \iff \mathrm{P} = \mathrm{NP}$.

The P versus NP Problem

$\mathrm{P} = \mathrm{NP}$



$\mathrm{P} \neq \mathrm{NP}$



Two Types of Reductions

Many-one: one call.

```
def solve_L1(x):
    y = phi(x)
    return solve_L2(y)
```

Turing: many calls.

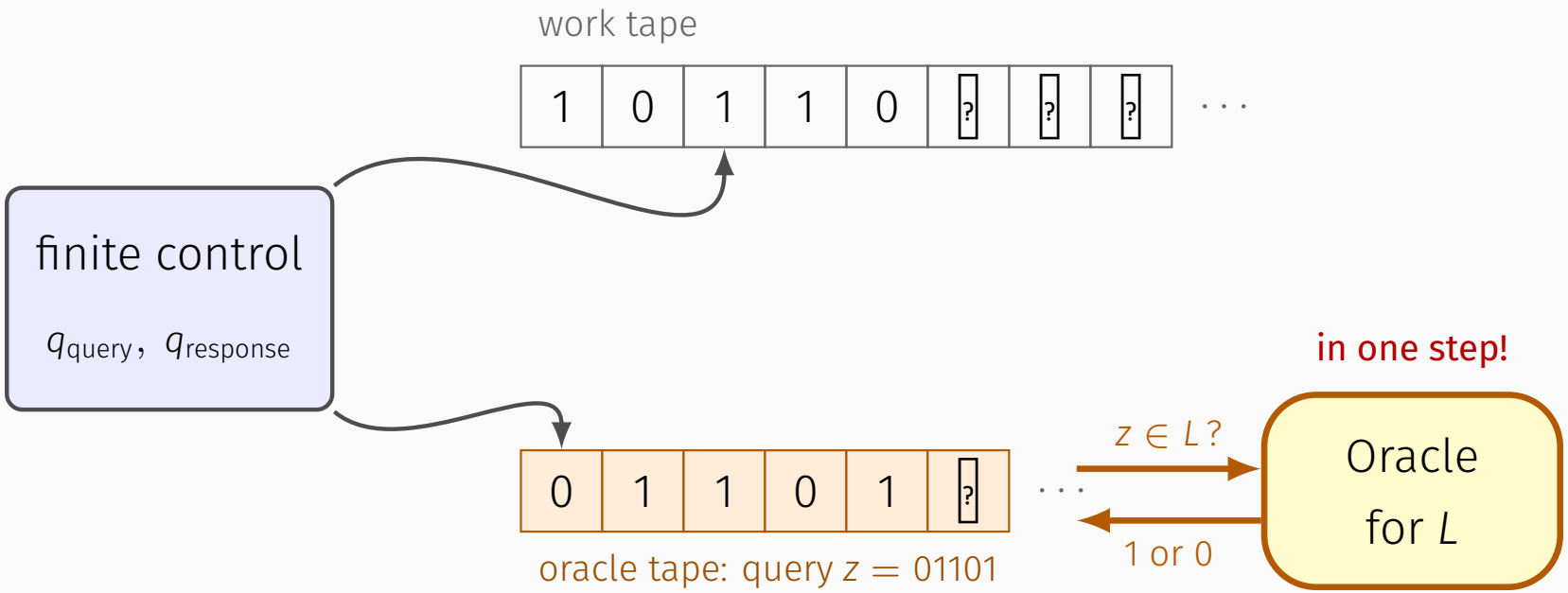
```
def solve_L1(x):
    ...
    a = solve_L2(...)
    ...
    b = solve_L2(...)
    ...
    return ...
```

Oracle Turing Machines: Formal Definition

Definition 5 (Oracle Turing machine)
An *oracle TM* M^L has an extra *oracle tape* and two special states $q_{\text{query}}, q_{\text{response}}$. On entering q_{query} with z on the oracle tape, in *one step*:

- z is replaced by 1 if $z \in L$, and by 0 otherwise;
- the oracle head moves to the leftmost cell;
- M^L enters q_{response}.

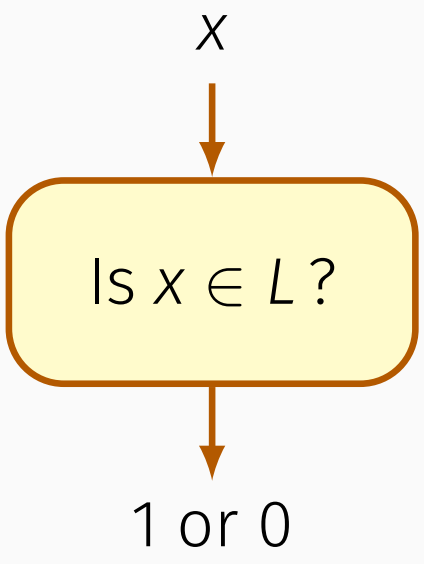
Oracle Turing Machine: The Picture



Write z on the oracle tape, enter $q_{\text{query}} \Longrightarrow$ the answer to "$z \in L$?" appears.

Decision Oracles vs. Function Oracles

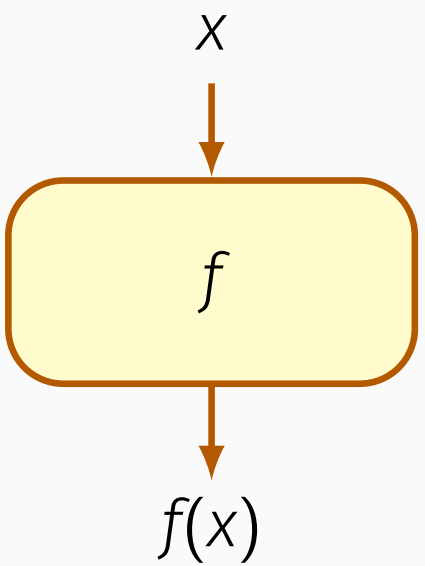
Decision oracle



output: 1 bit

e.g. $L = \mathrm{SAT}$: $\varphi \mapsto 1$

Function oracle



output: a string

e.g. $f =$ factoring: $391 \mapsto 17 \times 23$

Complexity Classes with Oracles

Definition 6
P^L is the class of languages decided by some *polynomial-time* oracle TM M^L.

Similarly, NP^L uses polynomial-time *nondeterministic* oracle TMs.

For a class $\mathcal{C}$ of oracles, $\mathrm{P}^{\mathcal{C}} = \bigcup_{L \in \mathcal{C}} \mathrm{P}^L$.

Turing Reduction

Definition 7 (Turing reduction)
$L_1 \leq_T L_2$ if some oracle TM M^{L_2} decides L_1.

Definition 8 (Cook reduction)
$L_1 \leq_T^p L_2$ if some *polynomial-time* oracle TM M^{L_2} decides L_1, i.e., $L_1 \in \mathrm{P}^{L_2}$.

Properties of Turing Reductions

Proposition 3

1. $L \leq_T \overline{L}$ *for every* L.
2. *If* L_1, L_2 *are decidable, then* $L_1 \leq_T L_2$.
3. *If* $L_1 \leq_T L_2$ *and* $L_2 \leq_T L_3$, *then* $L_1 \leq_T L_3$.
4. *If* $L_1 \leq_m L_2$, *then* $L_1 \leq_T L_2$.
5. *Some* L_1, L_2 *satisfy* $L_1 \not\leq_T L_2$ *and* $L_2 \not\leq_T L_1$.

Properties of Cook Reductions

Proposition 4

1. $L \leq_T^p \overline{L}$ *for every* L.
2. *If* $L_1 \in \mathrm{P}$, *then* $L_1 \leq_T^p L_2$ *for every* L_2.
3. *If* $L_1 \leq_T^p L_2$ *and* $L_2 \leq_T^p L_3$, *then* $L_1 \leq_T^p L_3$.
4. *If* $L_1 \leq_p L_2$, *then* $L_1 \leq_T^p L_2$.
5. *If* $L_1 \leq_T^p L_2$ *and* $L_2 \in \mathrm{P}$, *then* $L_1 \in \mathrm{P}$.

Summary: Reductions

- *Many-one* ($\leq_m$): one call. *Turing* ($\leq_T$): many calls.
- *Karp* ($\leq_p$) and *Cook* ($\leq_T^p$): their poly-time versions.
- NP-hard and NP-complete are defined via Karp reductions.